

Bloc 2 MVC PHP TD5

Contexte	2
Exercice 1	2
Exercice 2	3
Exercice 3	6
Exercice 4	7
Exercice 5	8
Exercice 7	10
Exercice 8	13
Exercice 9	14
Conclusion	15

Contexte

Ce **TD** est la suite du **TD4** sur l'architecture **MVC**. On continue le site de covoiturage : on change la page d'accueil pour passer par "**index.php**", on sécurise l'affichage des vues et on rend les vues modulaires (en-tête et pied de page communs).

(Les fichiers projet seront disponibles dans le portfolio sur GitHub).

Exercice 1

Rappel du fonctionnement du **MVC** sur l'exemple de l'action **read** :

- Le client demande la page "**routeur.php?action=read&immat=AA-000-AA**".
- Le routeur récupère l'action avec **\$_GET['action']** et appelle **ControllerVoiture::read()**.
- Le contrôleur récupère l'immatriculation dans l'URL et demande la voiture au modèle avec **getVoitureByImmat()**.
- Le modèle fait la requête préparée dans la **BDD** et renvoie un objet **ModelVoiture** (ou **false**).
- Le contrôleur met la voiture dans **\$v** et fait le **require** de la vue "**detail.php**", qui écrit la page **HTML** envoyée au client.

Exercice 2

Pour ne plus dépendre du dossier courant, on va utiliser des chemins absolus.

Création de la classe **File** dans “**lib/File.php**” avec la méthode statique

“**build_path()**” :

```
<?php
class File {

    public static function build_path($path_array) {
        // $ROOT_FOLDER (sans slash à la fin) vaut
        // "C:\xampp\htdocs\public_html\PHP\TD5"
        $ROOT_FOLDER = "C:\\xampp\\htdocs\\public_html\\PHP\\TD5";
        return $ROOT_FOLDER. '/' . join('/', $path_array);
    }
}
?>
```

Par exemple **File::build_path(array("config", "Conf.php"))** renvoie

“**C:\xampp\htdocs\public_html\PHP\TD5/config/Conf.php**”.

On inclut “**File.php**” dans le routeur (à la main, car **build_path** n’est pas encore déclarée), puis on modifie tous les **require** pour qu’ils utilisent **build_path** :

```
<?php
require_once '../lib/File.php';
require_once File::build_path(array("controller", "ControllerVoiture.php"));
// On récupère l'action passée dans l'URL
$action = $_GET['action'];
// Appel de la méthode statique $action de ControllerVoiture
ControllerVoiture::$action();
?>
```

```

<?php
require_once File::build_path(array("model", "ModelVoiture.php")); // chargement du modèle

class ControllerVoiture {

    public static function readAll() {
        $tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la BD
        require File::build_path(array("view", "voiture", "list.php")); // "redirige" vers la vu
    }

    public static function read() {
        $immat = $_GET['immat'];
        $v = ModelVoiture::getVoitureByImmat($immat);
        if ($v === false) {
            require File::build_path(array("view", "voiture", "error.php"));
        } else {
            require File::build_path(array("view", "voiture", "detail.php"));
        }
    }
}

```

```

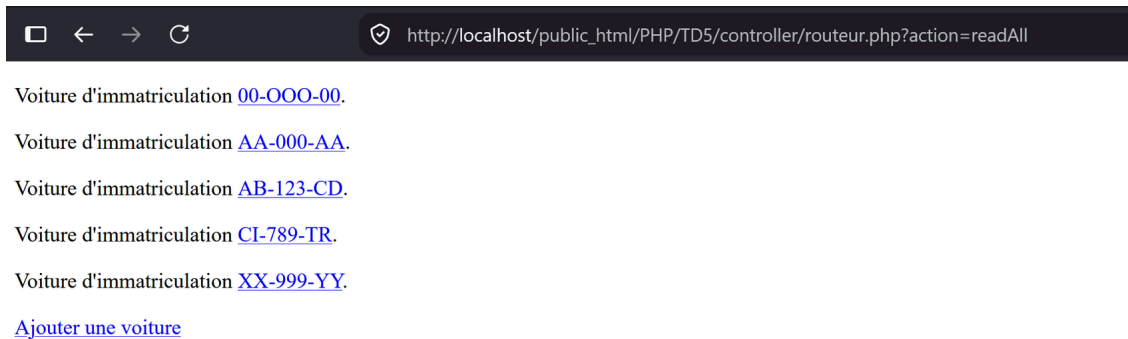
<?php

require_once File::build_path(array("config", "Conf.php"));

class Model {

```

On vérifie que le site marche toujours avec
“controller/routeur.php?action=readAll” :



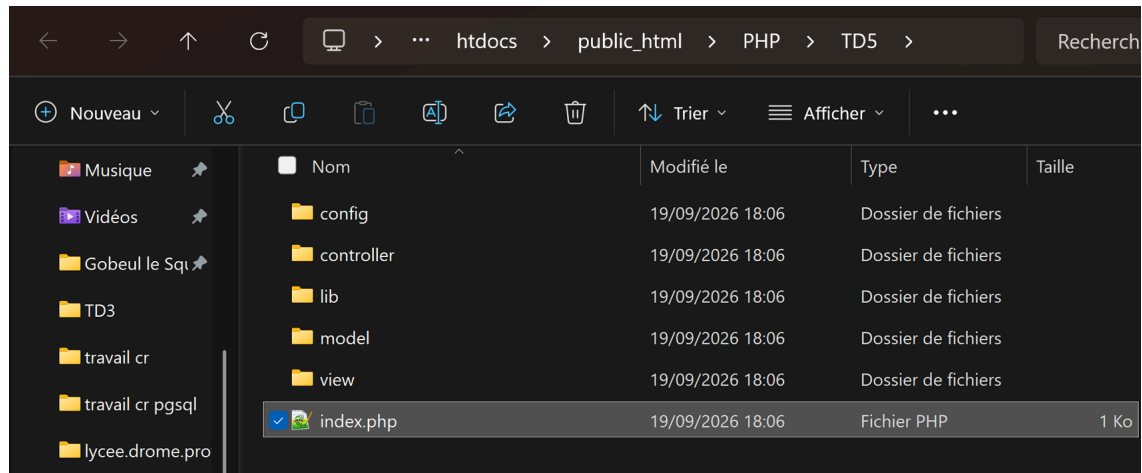
On veut que la page d’accueil soit **“index.php”**. Création de ce fichier à la racine du site, le **require** de **“File.php”** passe du routeur vers **“index.php”** :

```

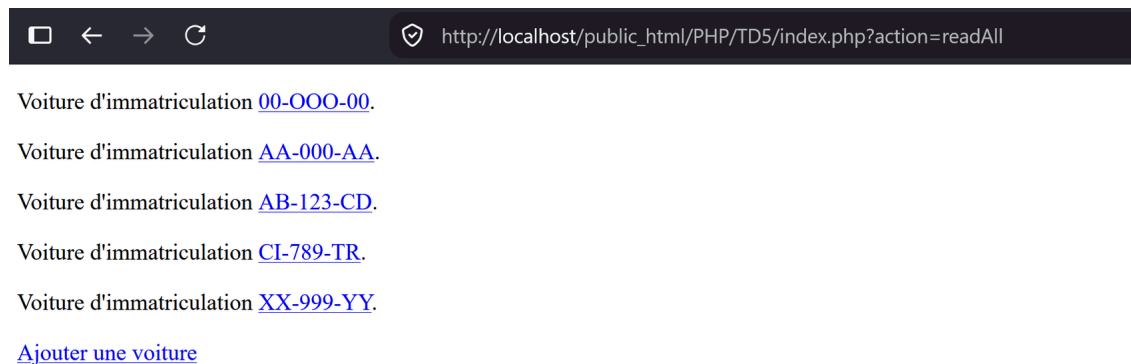
<?php
require_once 'lib/File.php';

require File::build_path(array("controller", "routeur.php"));
?>

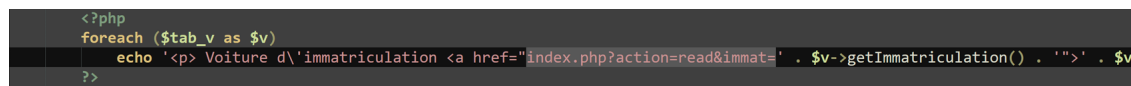
```



Test avec “**index.php?action=readAll**” :



On change aussi toutes les URLs des vues pour qu’elles pointent sur “**index.php**” à la place de “**controller/routeur.php**” :



Exercice 3

Pour que le site soit portable (Windows utilise des “\” et Linux des “/”), on utilise la constante magique `__DIR__` et `DIRECTORY_SEPARATOR` dans “`index.php`” :

```
<?php
// DS contient le slash des chemins de fichiers, c'est-à-dire '/' sur Linux et '\' sur Windows
$DS = DIRECTORY_SEPARATOR;
// __DIR__ est une constante "magique" de PHP qui contient le chemin du dossier courant
require_once __DIR__ . $DS . "lib" . $DS . "File.php";

require File::build_path(array("controller", "routeur.php"));
?>
```

Et dans “`build_path()`”, comme `__DIR__` renvoie le dossier “`lib`”, on remonte d’un dossier avec “`..`” :

```
<?php
class File {

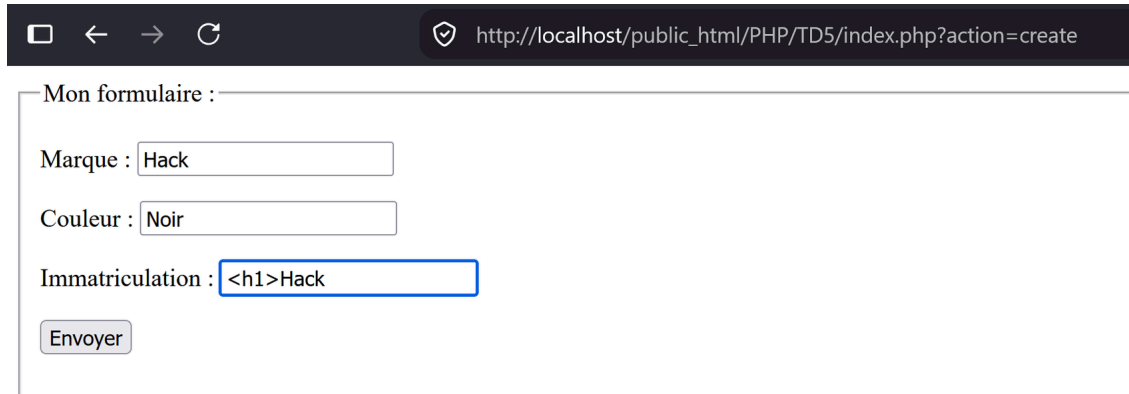
    public static function build_path($path_array) {
        // DS contient le slash des chemins de fichiers, c'est-à-dire '/' sur Linux et '\' sur Windows
        $DS = DIRECTORY_SEPARATOR;
        $ROOT_FOLDER = __DIR__ . $DS . "..";
        return $ROOT_FOLDER . $DS . join($DS, $path_array);
    }
}
?>
```

On reteste le site :



Exercice 4

On crée une voiture avec l'immatriculation "<h1>Hack" grâce à l'action **create** :



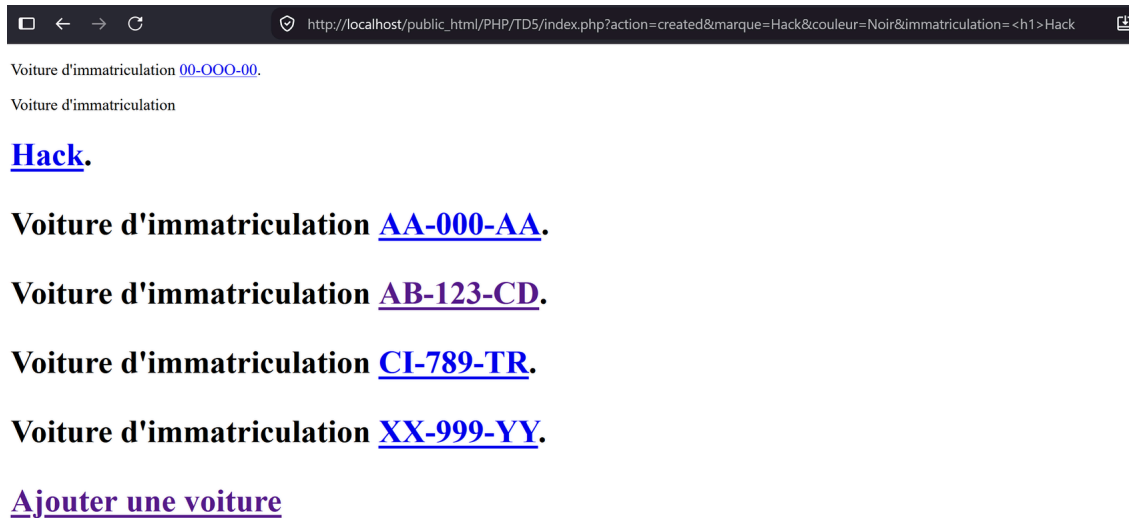
Mon formulaire :

Marque :

Couleur :

Immatriculation :

Résultat dans la liste des voitures :



Voiture d'immatriculation 00-000-00.

Voiture d'immatriculation

Hack.

Voiture d'immatriculation AA-000-AA.

Voiture d'immatriculation AB-123-CD.

Voiture d'immatriculation CI-789-TR.

Voiture d'immatriculation XX-999-YY.

Ajouter une voiture

Tout le texte après l'immatriculation s'affiche en gros. On regarde le code source de la page (Ctrl+U) :



```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title>Liste des voitures</title>
6   </head>
7   <body>
8     <p> Voiture d'immatriculation <a href="index.php?action=read&immat=00-000-00">00-000-00</a> </p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=<h1>Hack"><h1>Hack</a> </p><p>
9     Voiture d'immatriculation <a href="index.php?action=read&immat=AA-000-AA">AA-000-AA</a> </p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=AB-123-CD">AB-123-CD</a> </p><p> Voiture
10    d'immatriculation <a href="index.php?action=read&immat=CI-789-TR">CI-789-TR</a> </p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=XX-999-YY">XX-999-YY</a> </p> <p><a
11    href="index.php?action=create">Ajouter une voiture</a></p>
12  </body>
13 </html>
```

Question : « Que se passe-t-il si l'utilisateur a rentré du code HTML à la place d'une immatriculation ? »

Réponse : L'immatriculation est comprise comme du code **HTML** : la balise **<h1>** n'est jamais fermée, donc le navigateur l'interprète et tout ce qui suit s'affiche comme un titre. C'est dangereux car on pourrait aussi injecter du **JavaScript** (faille **XSS**).

Exercice 5

On applique la fonction **htmlspecialchars** à toutes les variables **PHP** des vues, en passant par des variables temporaires :

```
<?php
foreach ($tab_v as $v) {
    $vImmatriculation = htmlspecialchars($v->getImmatriculation());
    echo '<p> Voiture d\'immatriculation <a href="index.php?action=read&immat=' . $vImmatriculation . '"> . $vImmatriculation . "</a>";
}
?>
```

```
<?php
$vImmatriculation = htmlspecialchars($v->getImmatriculation());
$vMarque = htmlspecialchars($v->getMarque());
$vCouleur = htmlspecialchars($v->getCouleur());

echo "<p>Voiture " . $vImmatriculation . " de la marque " . $vMarque . " (couleur " . $vCouleur . ")</p>";
?>
```

La voiture “**<h1>Hack**” s'affiche maintenant correctement :



Voiture d'immatriculation [00-000-00.](#)

Voiture d'immatriculation [<h1>Hack.](#)

Voiture d'immatriculation [AA-000-AA.](#)

Voiture d'immatriculation [AB-123-CD.](#)

Voiture d'immatriculation [CI-789-TR.](#)

Voiture d'immatriculation [XX-999-YY.](#)

[Ajouter une voiture](#)

Dans le code source, les chevrons ont bien été échappés en **<** et **>** :

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>liste des voitures</title>
6 </head>
7 <body>
8 <p> Voiture d'immatriculation <a href="index.php?action=read&immat=00-000-00">00-000-00</a></p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=<h1>Hack"><h1>Hack</
a></p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=AA-000-AA">AA-000-AA</a></p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=AB-123-CD">AB-123-CD</a></p><p>
Voiture d'immatriculation <a href="index.php?action=read&immat=CI-789-TR">CI-789-TR</a></p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=XX-999-YY">XX-999-YY</a></p> <p><a
href="index.php?action=create">Ajouter une voiture</a></p>
9 </body>
10 </html>
11
```

Échappement des URLs

On crée une voiture d'immatriculation “&immat=h” avec l'action **create** :

Mon formulaire :

Marque :

Couleur :

Immatriculation :

Voiture d'immatriculation [&immat=h](#).

Voiture d'immatriculation [00-000-00](#).

Voiture d'immatriculation [<h1>Hack](#).

Voiture d'immatriculation [AA-000-AA](#).

Voiture d'immatriculation [AB-123-CD](#).

Voiture d'immatriculation [CI-789-TR](#).

Voiture d'immatriculation [XX-999-YY](#).

[Ajouter une voiture](#)

Quand on clique sur le lien de cette voiture, la page de détail ne marche pas :

Erreur : aucune voiture ne correspond à cette immatriculation.

[Retour à la liste](#)

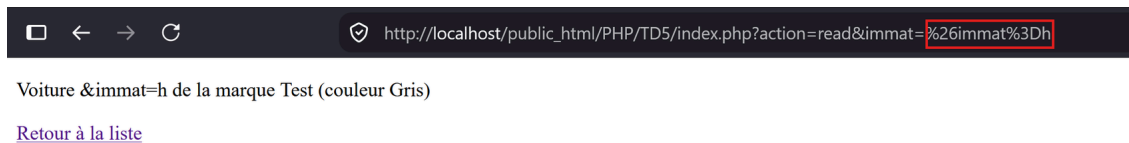
Question : « Pourquoi le lien ne marche pas ? »

Réponse : Le “&” de l’immatriculation est compris comme un séparateur de paramètres dans la query string. L’URL devient “...&immat=&immat=h”, donc PHP récupère **immat = “h”** (la dernière valeur) et ne trouve aucune voiture avec cette immatriculation.

On encode l’immatriculation avec **rawurlencode** dans “**list.php**”. Attention, il faut deux variables : une échappée pour le **HTML** et une encodée pour l’URL :

```
<?php
foreach ($tab_v as $v) {
    $vImmatriculation = htmlspecialchars($v->getImmatriculation());
    $vImmatriculationURL = rawurlencode($v->getImmatriculation());
    echo '<p> Voiture d\'immatriculation <a href="index.php?action=read&immat=' . $vImmatriculationURL . '>' .
}
?>
```

Le lien remarque, “&” et “=” sont encodés en **%26** et **%3D** :



The screenshot shows a web browser address bar with the URL: `http://localhost/public_html/PHP/TD5/index.php?action=read&immat=%26immat%3Dh`. The `%26` and `%3D` are highlighted with a red box. Below the address bar, the page content reads: "Voiture &immat=h de la marque Test (couleur Gris)" and a link "Retour à la liste".

Exercice 7

(Il n’y a pas d’exercice 6 dans le sujet).

Création de la vue générique “**view/view.php**”, elle écrit l’en-tête et le pied de page communs et charge la vue dont le nom est dans **\$view** :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title><?php echo $pagetitle; ?></title>
  </head>
  <body>
    <?php
    // Si $controleur='voiture' et $view='list',
    // alors $filepath="/chemin_du_site/view/voiture/list.php"
    $filepath = File::build_path(array("view", $controller, "$view.php"));
    require $filepath;
    ?>
  </body>
</html>
```

On supprime le header et le footer dans les vues existantes, par exemple “list.php” :

```
<h1>Liste des voitures :</h1>
<?php
foreach ($tab_v as $v) {
    $vImmatriculation = htmlspecialchars($v->getImmatriculation());
    $vImmatriculationURL = rawurlencode($v->getImmatriculation());
    echo ' <p> Voiture d\'immatriculation <a href="index.php?action=read&immat=' . $vImmatriculationURL . '>' . $vImmatr:
}
?>
<p><a href="index.php?action=create">Ajouter une voiture</a></p>
```

Dans l’action **readAll**, on initialise **\$controller**, **\$view** et **\$pagetitle** puis on fait un **require** de “view.php” :

```
public static function readAll() {
    $tab_v = ModelVoiture::getAllVoitures(); //appel au modèle pour gerer la B
    $controller = 'voiture';
    $view = 'list';
    $pagetitle = 'Liste des voitures';
    require File::build_path(array("view", "view.php"));
}
```

Test de l’action **readAll** :

http://localhost/public_html/PHP/TD5/index.php?action=readAll

Liste des voitures :

Voiture d'immatriculation [&immat=h.](#)

Voiture d'immatriculation [00-000-00.](#)

Voiture d'immatriculation [<h1>Hack.](#)

Voiture d'immatriculation [AA-000-AA.](#)

Voiture d'immatriculation [AB-123-CD.](#)

Voiture d'immatriculation [CI-789-TR.](#)

Voiture d'immatriculation [XX-999-YY.](#)

[Ajouter une voiture](#)

Le code source généré est correct (une seule balise **<html>**, **<head>** et **<body>**) :

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title>liste des voitures</title>
6   </head>
7   <body>
8     <h1>liste des voitures :</h1>
9     <p> Voiture d'immatriculation <a href="index.php?action=read&immat=%26immat%30h">8amp;immat=h</a>.</p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=00-000-00">00-000-00</a>.</p><p>
10    Voiture d'immatriculation <a href="index.php?action=read&immat=55h1338Hack">55h1338Hack</a>.</p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=AA-000-AA">AA-000-AA</a>.</p><p> Voiture
11    d'immatriculation <a href="index.php?action=read&immat=00-101-01">00-101-01</a>.</p><p> Voiture d'immatriculation <a href="index.php?action=read&immat=C1-789-10">C1-789-10</a>.</p><p> Voiture
12    d'immatriculation <a href="index.php?action=read&immat=XX-999-YY">XX-999-YY</a>.</p><p><a href="index.php?action=create">Ajouter une voiture</a></p>
13   </body>
14 </html>
```

On modifie de la même façon les autres actions (**read**, **create**) :

```
public static function read() {
    $immat = $_GET['immat'];
    $v = ModelVoiture::getVoitureByImmat($immat);
    $controller = 'voiture';
    if ($v === false) {
        $view = 'error';
        $pagetitle = 'Erreur';
    } else {
        $view = 'detail';
        $pagetitle = 'Détail de la voiture';
    }
    require File::build_path(array("view", "view.php"));
}

public static function create() {
    $controller = 'voiture';
    $view = 'create';
    $pagetitle = 'Créer une voiture';
    require File::build_path(array("view", "view.php"));
}
```

Exercice 8

Grâce à la vue générique, on peut ajouter un menu et un footer à toutes les pages en une seule fois dans “**view.php**” :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title><?php echo $pagetitle; ?></title>
  </head>
  <body>
    <nav>
      <a href="index.php?action=readAll">Voitures</a> |
      <a href="index.php?action=readAll&controller=utilisateur">Utilisateurs</a> |
      <a href="index.php?action=readAll&controller=trajet">Trajets</a>
    </nav>
    <?php
    // Si $controleur='voiture' et $view='list',
    // alors $filepath="/chemin_du_site/view/voiture/list.php"
    $filepath = File::build_path(array("view", $controller, "$view.php"));
    require $filepath;
    ?>
    <p style="border: 1px solid black;text-align:right;padding-right:1em;">
      Site de covoiturage de Romain Aubrée
    </p>
  </body>
</html>
```

Résultat :



Exercice 9

Création de la vue “**created.php**” qui affiche le message puis fait un **require** de “**list.php**” (elle ne fait plus que deux lignes) :

```
<p>La voiture a bien été créée !</p>
<?php require File::build_path(array("view", "voiture", "list.php")); ?>
```

Dans l’action **created**, on initialise **\$tab_v** pour que la liste puisse être affichée sous le message :

```
public static function created() {
    $marque = $_GET['marque'];
    $couleur = $_GET['couleur'];
    $immatriculation = $_GET['immatriculation'];

    $v = new ModelVoiture($marque, $couleur, $immatriculation);
    $v->save();

    $tab_v = ModelVoiture::getAllVoitures();
    $controller = 'voiture';
    $view = 'created';
    $pagetitle = 'Voiture créée';
    require File::build_path(array("view", "view.php"));
}
```

Test en créant une voiture avec le formulaire :

Voitures | Utilisateurs | Trajets

Mon formulaire :

Marque :

Couleur :

Immatriculation :



Conclusion

Ce **TD** nous a permis d'avoir une vraie page d'accueil "**index.php**" avec des chemins portables, de sécuriser nos vues contre les injections **HTML** (**htmlspecialchars**, **rawurlencode**) et de rendre les vues modulaires avec une vue générique qui contient le menu et le footer.